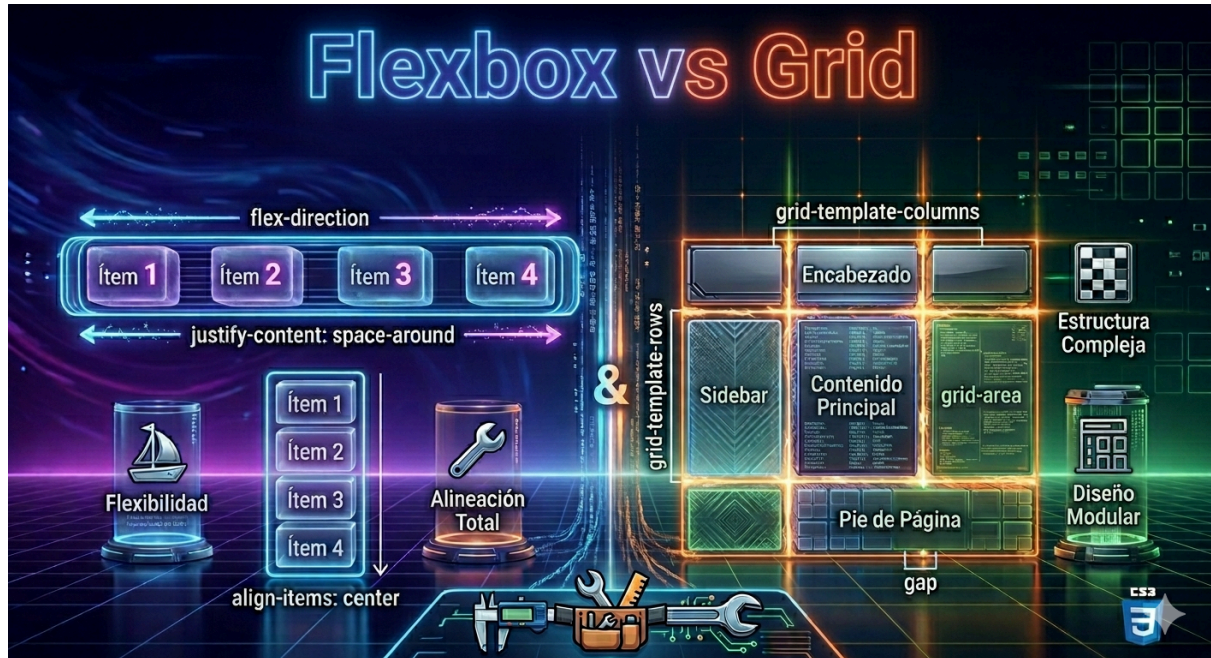


GUIA DE LAYOUT FLEXBOX Y GRID

DESARROLLO DE APLICACIONES WEB - DAW135



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Introducción

El siguiente documento es una guía visual y práctica para dominar los dos sistemas de layout más poderosos del CSS moderno. Aprenderás a construir una página real usando **Flexbox** para alinear elementos en una dimensión y **Grid** para organizar estructuras de página en dos dimensiones.



Barra de navegación

Usando Flexbox para distribuir espacio horizontalmente.



Tarjetas flexibles

Usando Flexbox con control individual de cada hijo.



Layout de página

Usando Grid con áreas nombradas para estructurar el contenido.

Requisitos previos

- Un editor de código como **Visual Studio Code**
- Un **navegador web** actualizado
- Conocimientos básicos de **HTML y CSS**

Estructura de carpetas del proyecto

Crea la siguiente estructura antes de escribir cualquier línea de código. Mantener los estilos separados en distintos archivos CSS es una buena práctica que facilita el mantenimiento y la lectura del proyecto.

```
Guia_Flexbox_Grid/
```

```
|-- index.html
```

```
`-- css/
```

```
    |-- base.css
```

```
    |-- flexbox.css
```

```
    |-- grid.css
```

¿Por qué separar los archivos CSS? Separar los estilos en **base.css**, **flexbox.css** y **grid.css** permite mantener cada responsabilidad aislada. Así es más fácil modificar o reutilizar cada parte sin afectar al resto del proyecto.

Paso 1: Crear el archivo HTML principal

El archivo `index.html` es la base de toda la guía. Aquí se declara la estructura semántica del documento y se enlazan los tres archivos CSS que escribiremos a continuación.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Guía Flexbox y Grid</title>
  <!-- Estilos generales -->
  <link rel="stylesheet" href="css/base.css">
  <!-- Estilos específicos de Flexbox -->
  <link rel="stylesheet" href="css/flexbox.css">
  <!-- Estilos específicos de Grid -->
  <link rel="stylesheet" href="css/grid.css">
</head>
<body>
  <header class="page-header">
    <h1>Guía: Flexbox y Grid</h1>
    <p class="lead">Aprenderemos a construir layouts con ejemplos pequeños y
claros.</p>
  </header>
  <main class="guide-content">
    <!-- Aquí irán las secciones -->
  </main>
</body>
</html>
```

Enlazar varios archivos CSS

Cada `<link>` carga un archivo CSS por separado. El orden importa: los estilos base deben ir primero.

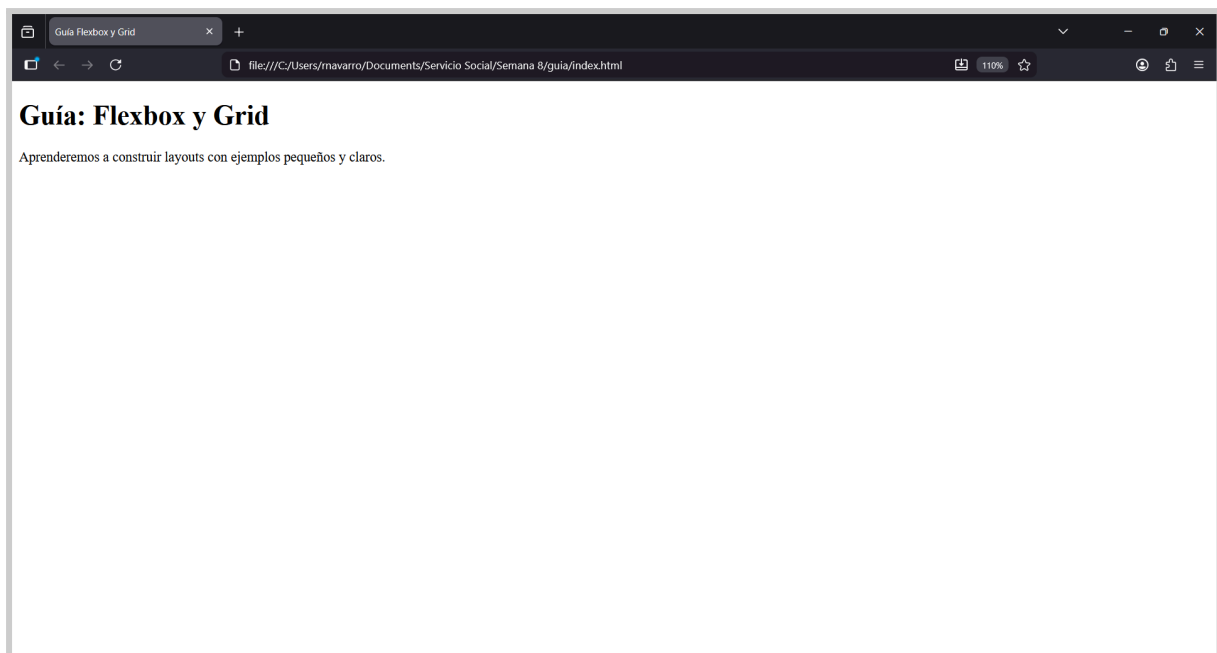
Meta viewport

Esta etiqueta es fundamental para que la página se adapte correctamente en pantallas de teléfonos y tabletas.

Base ordenada

Usar `header` y `main` con clases claras desde el inicio facilita escalar el proyecto sin desorden.

Resultado paso 1:



Paso 2: Agregar la navegación con Flexbox

Dentro de `<main class="guide-content">`, agrega el siguiente bloque. Este ejemplo muestra el caso de uso más clásico de Flexbox: una barra de navegación donde el logo está a un lado y los enlaces al otro. El contenedor `nav.menu-flex` será el **flex container** y sus hijos directos serán los **flex items**.

```
<section class="demo-section" id="paso-1">
  <div class="section-copy">
    <p class="step-label">Paso 1</p>
    <h2>Flexbox: Barra de navegación</h2>
    <p>Este ejemplo muestra cómo repartir espacio entre un logo y un grupo de
    enlaces.</p>
  </div>
  <nav class="menu-flex" aria-label="Navegación de ejemplo">
    <span class="logo">Mi Sitio</span>
    <div class="links">
      <a href="#paso-1">Inicio</a>
      <a href="#paso-2">Tarjetas</a>
      <a href="#paso-3">Layout Grid</a>
    </div>
  </nav>
</section>
```

El nav como flex container

El elemento `nav.menu-flex` recibirá `display: flex`. Esto convierte a sus hijos directos (`.logo` y `.links`) en flex items automáticamente.

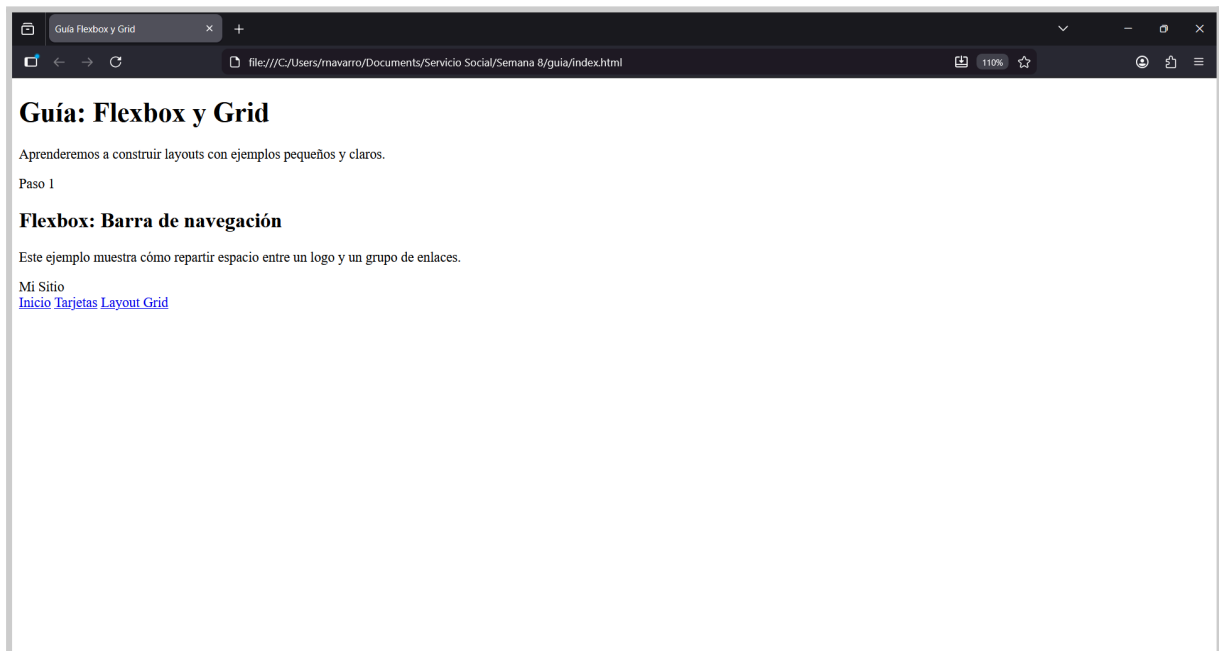
Agrupación de enlaces

Los enlaces están dentro de un `div.links` secundario. Esto permite tratarlos como un solo bloque desde la perspectiva del nav, facilitando el control del espacio con `justify-content: space-between`.

Distribución horizontal

Flexbox distribuirá el espacio entre el logo y el grupo de enlaces de forma horizontal. El valor `space-between` llevará cada elemento a los extremos del contenedor.

Resultado paso 2:



Paso 3: Agregar tarjetas con Flexbox

Debajo del bloque de navegación, agrega esta segunda sección. El objetivo es ver cómo diferentes hijos pueden comportarse de forma distinta dentro del mismo flex container usando las propiedades `flex-grow` y `flex-basis`. La tarjeta destacada recibirá más espacio que las demás.

```
<section class="demo-section" id="paso-2">
  <div class="section-copy">
    <p class="step-label">Paso 2</p>
    <h2>Flexbox: Tarjetas con control individual</h2>
    <p>Una tarjeta crece más que las demás para comparar comportamientos.</p>
  </div>
  <div class="cards-flex" role="list" aria-label="Tarjetas de ejemplo">
    <article class="card destacada" role="listitem">
```

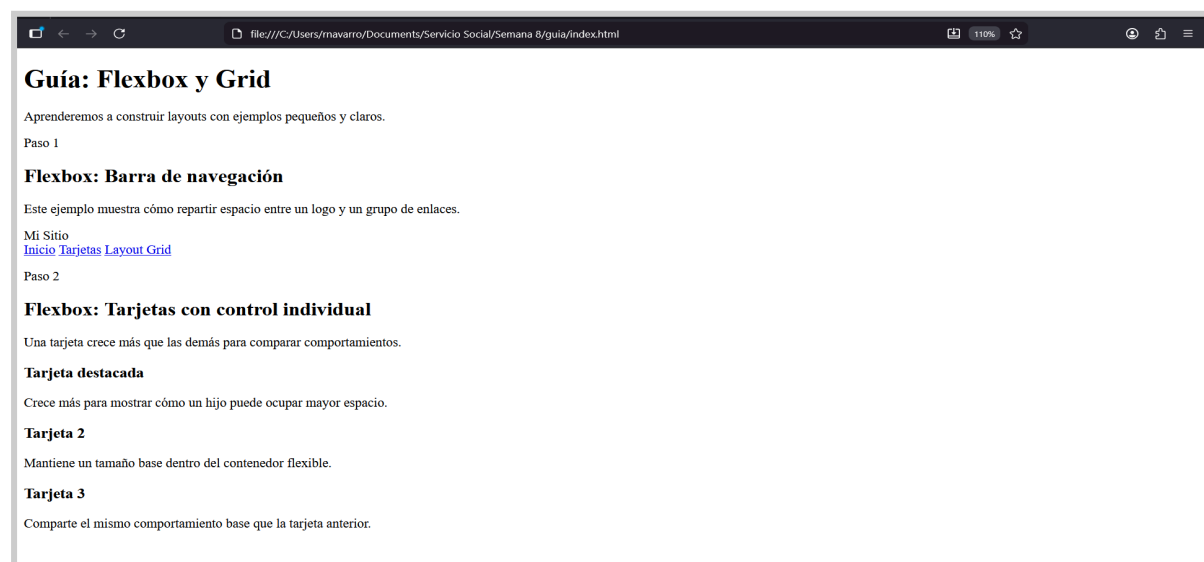
```

    <h3>Tarjeta destacada</h3>
    <p>Crece más para mostrar cómo un hijo puede ocupar mayor espacio.</p>
  </article>
  <article class="card" role="listitem">
    <h3>Tarjeta 2</h3>
    <p>Mantiene un tamaño base dentro del contenedor flexible.</p>
  </article>
  <article class="card" role="listitem">
    <h3>Tarjeta 3</h3>
    <p>Comparte el mismo comportamiento base que la tarjeta anterior.</p>
  </article>
</div>
</section>

```

flex-wrap	flex-grow	flex-basis
Permite que las tarjetas bajen a una nueva fila cuando no hay suficiente espacio horizontal, evitando el desbordamiento del contenido.	Controla cuánto espacio extra puede absorber un elemento. La tarjeta destacada usa flex-grow: 2, el doble que las demás.	Define el tamaño de partida antes de que el espacio disponible se distribuya. Funciona como un ancho mínimo de referencia.

Resultado paso 3:



Paso 4: Agregar un layout con Grid

Debajo de las tarjetas, agrega esta tercera sección. CSS Grid está diseñado para estructuras de página completas: header, sidebar, contenido principal y footer. Con `grid-template-areas`, cada zona recibe un nombre semántico que facilita la lectura del código y la reorganización del layout.

```
<section class="demo-section" id="paso-3">
  <div class="section-copy">
    <p class="step-label">Paso 3</p>
    <h2>Grid: Layout de página</h2>
    <p>Este ejemplo organiza una página usando áreas nombradas.</p>
  </div>
  <div class="layout-grid" aria-label="Layout principal de ejemplo">
    <header>
      <h3>Header</h3>
      <p>Encabezado principal del sitio.</p>
    </header>
    <aside>
      <h3>Sidebar</h3>
      <p>Área secundaria para menú o recursos.</p>
    </aside>
    <div class="main-panel">
      <h3>Contenido principal</h3>
      <p>Zona principal donde vive la información más importante.</p>
    </div>
    <footer>
      <h3>Footer</h3>
      <p>Pie con enlaces y datos de cierre.</p>
    </footer>
  </div>
</section>
```

display: grid	grid-template-areas	grid-template-columns
Activa el modo Grid en el contenedor. A diferencia de Flexbox, Grid trabaja simultáneamente en filas y columnas.	Permite definir visualmente la estructura del layout usando nombres de área. El código se vuelve autodocumentado y muy fácil de modificar.	Define el ancho de cada columna. Con <code>minmax()</code> se establece un rango flexible que respeta un mínimo y un máximo.

Resultado paso 4:

Guía: Flexbox y Grid

Aprenderemos a construir layouts con ejemplos pequeños y claros.

Paso 1

Flexbox: Barra de navegación

Este ejemplo muestra cómo repartir espacio entre un logo y un grupo de enlaces.

Mi Sitio

[Inicio](#) [Tarjetas](#) [Layout Grid](#)

Paso 2

Flexbox: Tarjetas con control individual

Una tarjeta crece más que las demás para comparar comportamientos.

Tarjeta destacada

Crece más para mostrar cómo un hijo puede ocupar mayor espacio.

Tarjeta 2

Mantiene un tamaño base dentro del contenedor flexible.

Tarjeta 3

Comparte el mismo comportamiento base que la tarjeta anterior.

Paso 3

Grid: Layout de página

Este ejemplo organiza una página usando áreas nombradas.

Header

Encabezado principal del sitio.

Sidebar

Área secundaria para menú o recursos.

Contenido principal

Zona principal donde vive la información más importante.

Footer

Pie con enlaces y datos de cierre.

Paso 5: Crear los estilos generales

En `css/base.css`, escribe los estilos que aplican a toda la página. Las variables CSS (`:root`) centralizan los colores y facilitan los cambios globales sin tener que buscar valores repetidos en distintos archivos.

```
:root {  
  --bg-page: #f4f7fb;  
  --bg-card: #ffffff;  
  --bg-block: #3498db;  
  --text-main: #1f2933;  
  --text-soft: #52606d;  
  --text-light: #ffffff;  
  --border-soft: #d9e2ec;  
}
```

```
* { box-sizing: border-box; }
```

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  background-color: var(--bg-page);  
  color: var(--text-main);  
  line-height: 1.6;  
  padding: 20px;
```

```
}
```

```
.page-header,  
.guide-content { max-width: 980px; margin: 0 auto; }
```

```
.demo-section {  
  background-color: var(--bg-card);  
  border: 1px solid var(--border-soft);  
  border-radius: 14px;  
  padding: 24px;  
  margin-bottom: 28px;  
}
```

```
.layout-grid > *, .card {  
  padding: 20px;  
  background-color: var(--bg-block);  
  color: var(--text-light);  
  border-radius: 10px;  
}
```

```
.step-label {  
  text-transform: uppercase;  
  letter-spacing: 0.08em;  
  font-weight: bold;  
  color: #0f766e;  
}
```

Variables CSS con :root

Centraliza todos los colores y valores reutilizables. Si cambias `--bg-block`, el cambio se aplica en todos los lugares donde se usa esa variable automáticamente.

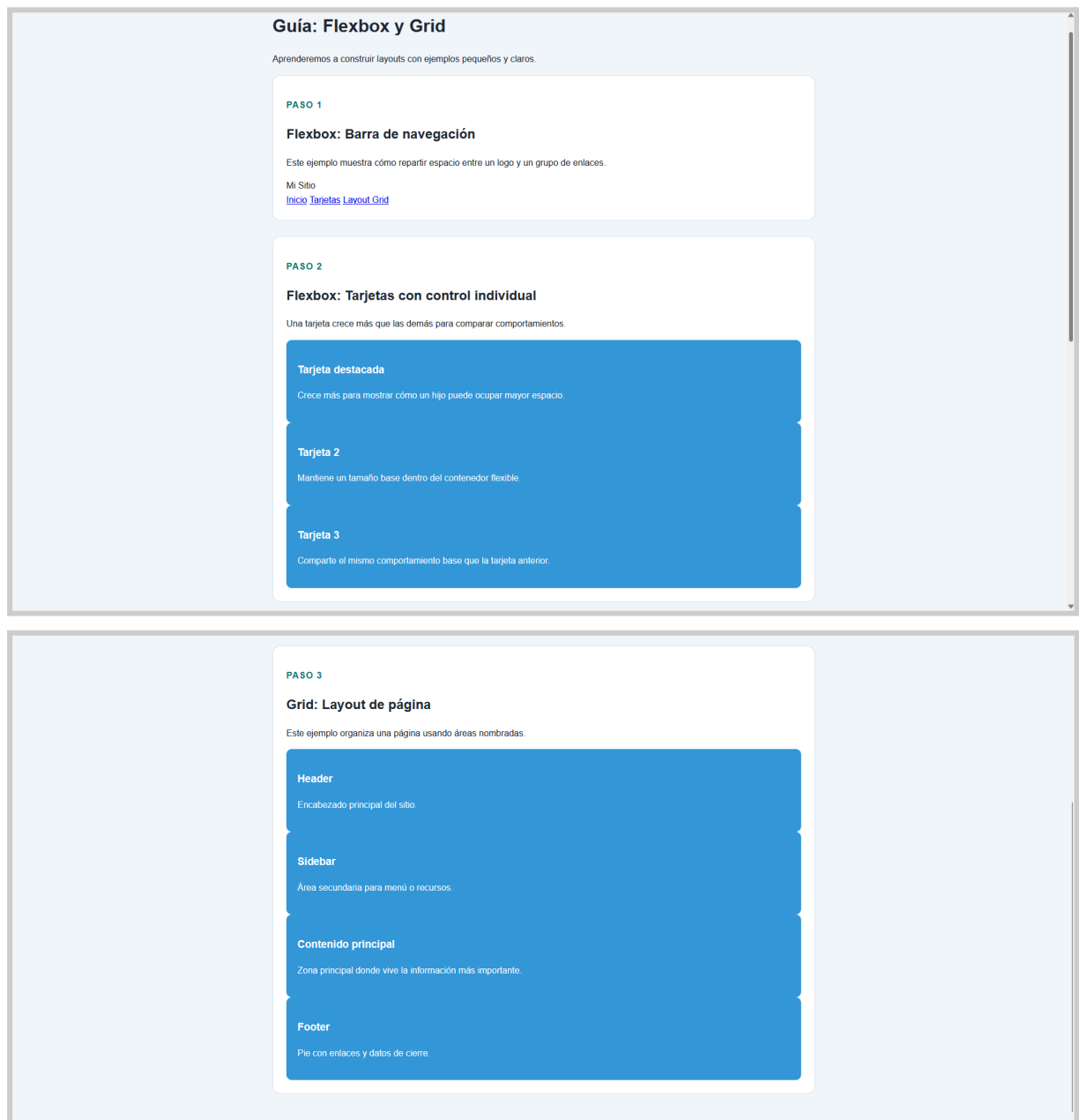
box-sizing: border-box

Hace que el padding y el borde se incluyan dentro del ancho declarado de cada elemento. Esto evita que los elementos se salgan de sus contenedores por cálculos inesperados.

Base visual consistente

Al definir márgenes, fuentes y colores en `base.css`, los archivos de Flexbox y Grid solo necesitan preocuparse por las propiedades de layout, sin mezclar responsabilidades.

Resultado paso 5:



Paso 6: Crear los estilos de Flexbox

En `css/flexbox.css`, escribe todos los estilos relacionados con la navegación y las tarjetas. Este archivo agrupa exclusivamente las propiedades Flexbox para que sea fácil de leer, comparar y modificar de forma independiente.

```
.menu-flex {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  flex-wrap: wrap;  
  gap: 12px 20px;
```

```
background-color: #2c3e50;
padding: 15px 18px;
border-radius: 10px;
}
```

```
.links {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
```

```
.menu-flex a {
  color: white;
  text-decoration: none;
  font-weight: bold;
}
```

```
.cards-flex {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
```

```
.card { flex: 1 1 180px; }
```

```
.card.destacada {
  flex: 2 1 260px;
  background-color: #1abc9c;
}
```

¿Qué pasa sin display: flex?

Si eliminas **display: flex** de **.menu-flex**, el logo y los enlaces se apilan verticalmente como bloques normales, perdiendo la distribución horizontal.

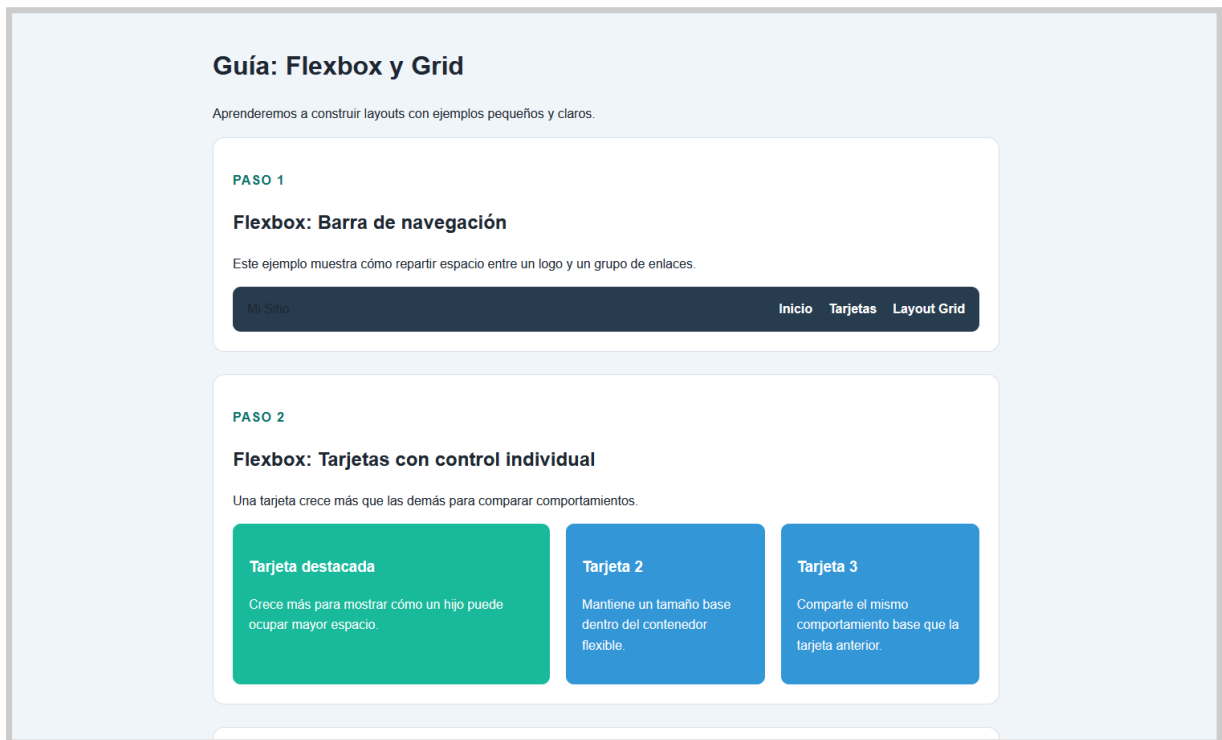
space-between vs center

Con **center**, el logo y los enlaces se agrupan al centro del contenedor. Con **space-between**, ocupan los extremos opuestos del nav.s.

flex: 1 vs flex: 2

Si la tarjeta destacada usa **flex: 1 1 260px**, su crecimiento será igual al de las demás. Con **flex: 2**, absorbe el doble del espacio disponible.

Resultado paso 6:



Paso 7: Crear los estilos de Grid

En `css/grid.css`, escribe los estilos que controlan el layout de la página. Las **áreas nombradas** de Grid hacen que la estructura del CSS sea prácticamente un mapa visual del diseño, lo cual facilita la colaboración y el mantenimiento del código a largo plazo.

```
.layout-grid {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  grid-template-columns: minmax(180px, 240px) minmax(0, 1fr);  
  gap: 15px;  
}
```

```
.layout-grid header { grid-area: header; }  
.layout-grid aside { grid-area: sidebar; }  
.main-panel { grid-area: main; }  
.layout-grid footer { grid-area: footer; }
```



Orden visual en grid-template-areas

Si cambias el orden de las áreas, el layout visual cambia aunque el orden del HTML permanezca igual. Grid separa la estructura del código de su presentación visual.



1fr vs minmax()

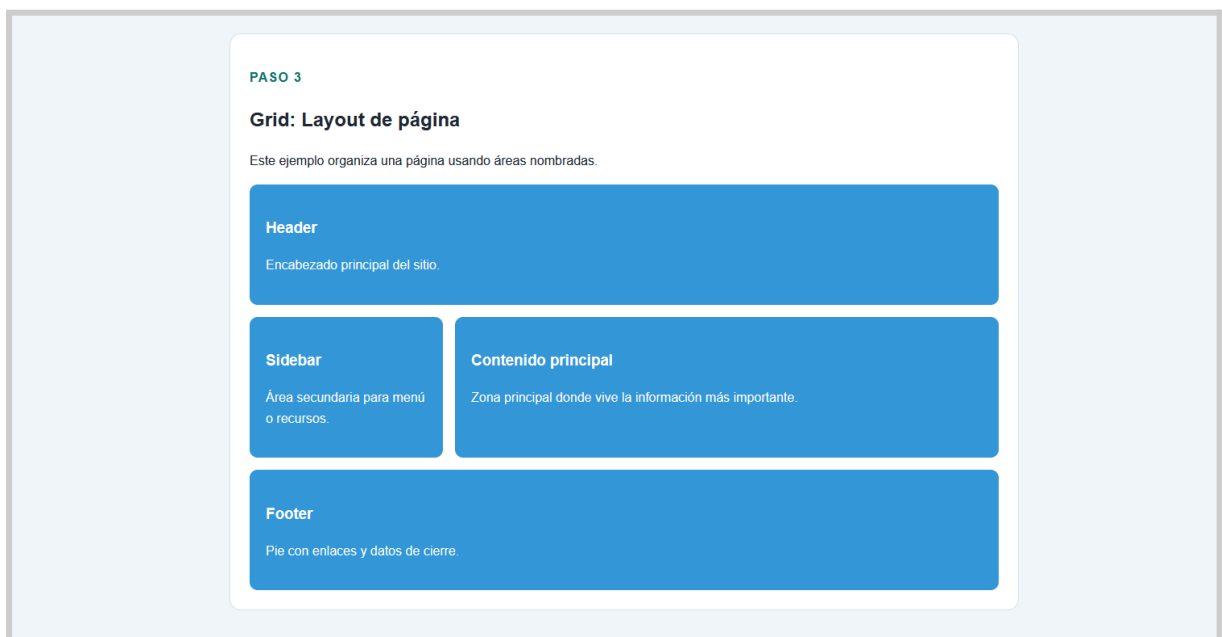
Con `1fr`, ambas columnas tendrán el mismo ancho. Con `minmax(180px, 240px)` el sidebar tiene un rango controlado, mientras que el main crece libremente.



Grid vs Flexbox para este layout

Con Flexbox necesitarías contenedores anidados y trucos adicionales. Grid resuelve el layout completo en dos dimensiones desde un solo contenedor, de forma más directa.

Resultado paso 7:



Paso 8: Adaptar la guía a pantallas pequeñas

Las **media queries** permiten cambiar el comportamiento del layout según el ancho disponible. En pantallas pequeñas, la navegación horizontal se convierte en vertical, y las columnas del Grid se apilan en una sola columna para evitar el desbordamiento y mejorar la legibilidad.

css/flexbox.css

```
@media (max-width: 700px) {  
  .menu-flex {  
    flex-direction: column;  
    align-items: flex-start;  
  }  
}
```

css/grid.css

```
@media (max-width: 760px) {  
  .layout-grid {  
    grid-template-areas:  
      "header"  
      "sidebar"  
      "main"  
      "footer";  
    grid-template-columns: 1fr;  
  }  
}
```

Flexbox en móvil

Cambiar **flex-direction** a **column** apila los elementos verticalmente. **align-items: flex-start** evita que los ítems se estiren al ancho completo del contenedor.

Grid en móvil

Al cambiar a **grid-template-columns: 1fr**, cada área ocupa el 100% del ancho disponible y se apila en el orden definido en **grid-template-areas**.

PASO 2

Flexbox: Tarjetas con control individual

Una tarjeta crece más que las demás para comparar comportamientos.

Tarjeta destacada
Crece más para mostrar cómo un hijo puede ocupar mayor espacio.

Tarjeta 2
Mantiene un tamaño base dentro del contenedor flexible.

Tarjeta 3
Comparte el mismo comportamiento base que la tarjeta anterior.

PASO 3

Grid: Layout de página

Este ejemplo organiza una página usando áreas nombradas.

Header
Encabezado principal del sitio.

Sidebar
Área secundaria para menú o recursos.

Contenido principal
Zona principal donde vive la información más importante.

Footer
Pie con enlaces y datos de cierre.

Paso 9: Comprobar el resultado

Abre el archivo **index.html** directamente en el navegador y verifica cada punto de la siguiente lista. Si algo no se ve como se describe, revisa el archivo CSS correspondiente al elemento que falla. Las herramientas de desarrollador del navegador (F12) son tu mejor aliado para inspeccionar el layout en tiempo real.

